AI+ Kuikly：7.5小时落地三端「多模态聊天 App」实战

原创 腾讯程序员 腾讯技术工程 Jun 30, 2026, 2:44 AM 贵州 147 listened



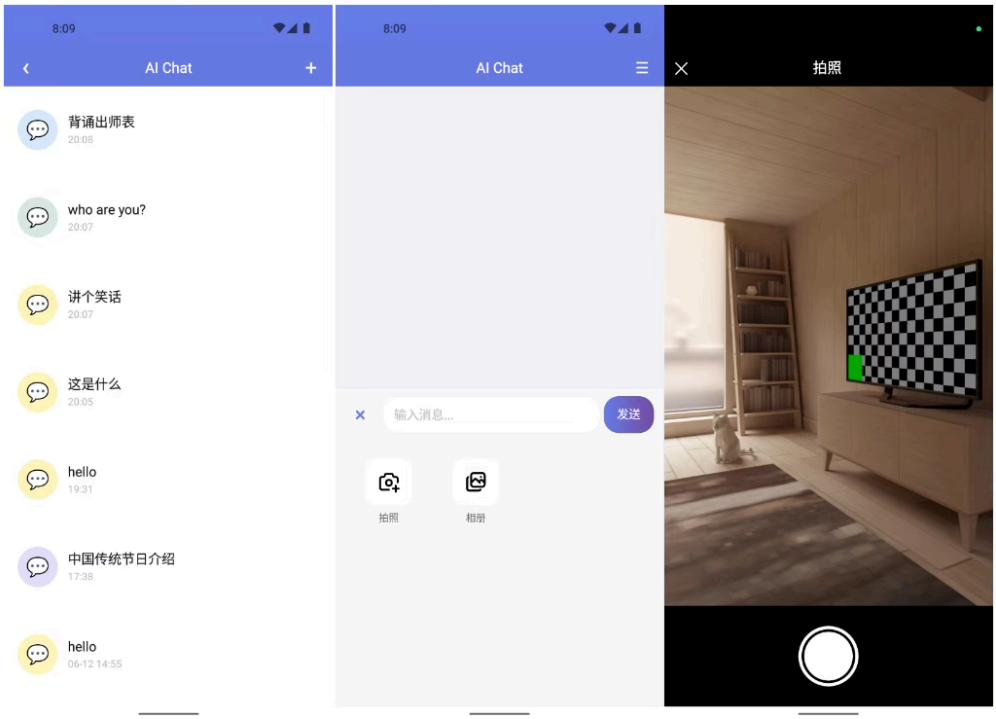
> Kuikly 是腾讯开源的高性能跨端框架，基于 Kotlin Multiplatform 技术，覆盖 Android、iOS、HarmonyOS、H5、微信小程序、Mac 六大平台，支撑业务日活用户超5亿。当 Kuikly 搭配真正懂它的 AI，开发会怎样——零手写代码，仅凭自然语言，7.5 小时交付一套支持 Android、iOS、鸿蒙三端的 AI 聊天 App。看 AI 如何调研组件、扩展原生模块、自行定位 Bug，感受为什么「AI + Kuikly」是当下客户端开发效率最高的组合之一。

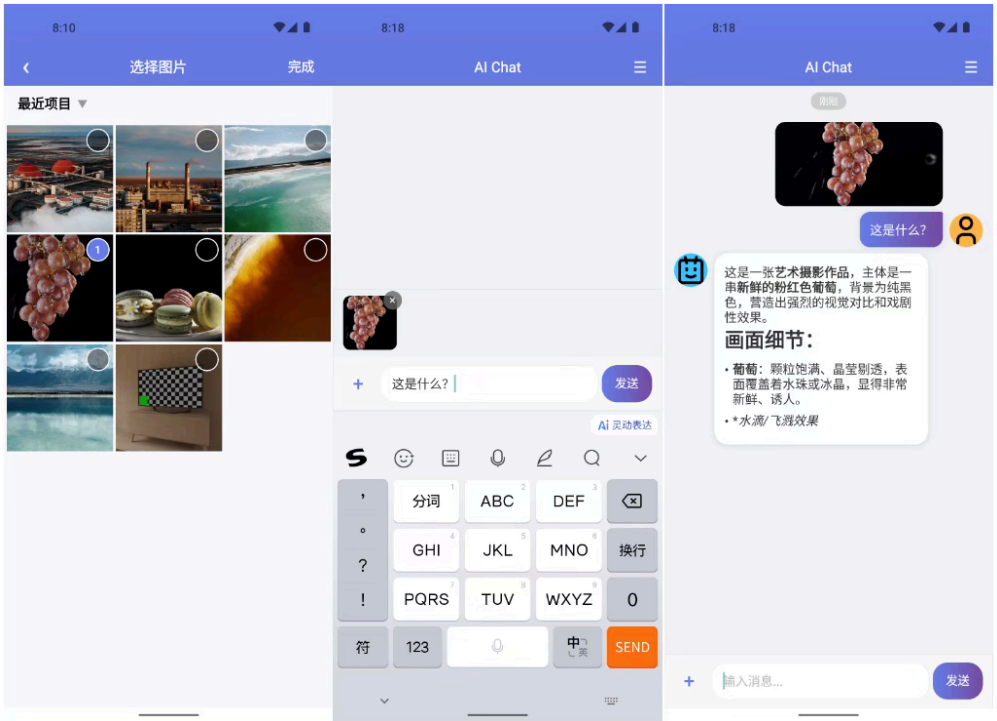
用 28 轮对话、740 字自然语言，生成约 3500 行代码，完成一套三端可运行的多模态 AI 聊天 App。全程零手写，不看代码，1 天交付。

放在传统开发里，同样的功能 iOS、Android、鸿蒙各写一遍，要 30 人天；就算用 Kuikly 手写，也得 7.5 人天。这次用 AI 辅助，实际只花了 7.5 小时。最终交付的 App 支持流式 Markdown、拍照识图、相册选取、SSE 长连接、本地会话管理，一套代码即可覆盖 Android、iOS、鸿蒙三端。

这不是“Vibe Coding”的玄学叙事，而是一次“AI + Kuikly 跨端框架”的实弹演习。Skills 和 Rules 让 AI 始终保持在正确的技术上下文中，组件库开箱即用、三端模板一键生成，这套基础设施支撑起了“AI + Kuikly”的协同效率。之前 搜狗输入法用 Spec Coding 把新页面开发从 3 天压到 1 天， QQ 音乐用 AI 智能转码实现 90%+ 代码采用率 ，效率红利正在被验证。

说到底，这次实验就想回答一个问题：纯靠对话，能把事情做到什么程度？一天之后，我有了答案。





指标	数值
总耗时	1天
对话轮数	28轮
代码总行数	≈3500
手写代码行数	≈0
复用社区组件	6个

下面是这一天的真实日记。

AI Coding 全过程

先把这一天的节奏拉成一条时间线：

时间	阶段
09:00 – 09:10	环境准备
09:10 – 10:20	需求分析与方案设计
10:20 – 11:10	编码实现
11:10 – 12:30	集成自测
14:00 – 17:30	迭代优化
17:30 – 18:00	验收复盘

09:00 - 09:10｜环境准备：先把 AI 开发环境搭起来

动手写代码前，得先把 AI 的“工作环境”备齐。

先跟着KuiklyUI GitHub 仓库的指引装好 IDE 和 Kuikly 插件。我本地早就配好了，直接跳过这一步，开始创建模板工程。按向导填写信息、一路点下一步，Android、iOS、鸿蒙三端的工程文件就准备好了。

> 觉得 Kuikly 有意思的话，欢迎给 KuiklyUI GitHub 仓库点个 ⭐ 关注，以后好找。

接着，按照官网AI编程的指引在工程里安装 Kuikly AI 的 Skills 和 Rules。

```
npx skills add Tencent-TDS/KuiklyUI-AI/skills
```

这一步很关键。Kuikly DSL 相对专有，通用大模型的训练语料覆盖并不充分，而 Skills 和 Rules 能把框架知识喂给模型，让 AI 像一个熟悉 Kuikly 的开发者一样工作。

到这里，Kuikly 的 AI 开发环境就算准备好了，然后我切到 AI Coding 的主场：CodeBuddy。

09:10 - 10:20｜需求分析与方案设计：先确定能复用什么，再决定自己写什么

这是一个完整 App 而非单个页面，所以我没让它直接开写，而是先用 CodeBuddy superpowers 插件里的 `brainstorming` 技能把需求拆清楚。

给它的第一条指令：

> /brainstorming 使用 Kuikly 实现一个多模态 AI 聊天 App，一码三端，支持 Android、iOS、鸿蒙。核心能力包括：发送文本/图片消息、拍照发图、相册选图、AI 流式回复、Markdown 渲染、打开消息中的网址、本地会话管理、历史会话恢复。优先使用 Kuikly 官方和社区已有组件。

这条 Prompt 写得比较完整，关键是两点：明确“一码三端”，不是只跑 Android 的 Demo；强调“优先使用已有组件”，避免它上来就造轮子。

组件查询

AI 随即调用查询第三方组件的技能，筛出一份匹配清单（`KuiklyChatUI`、`KuiklyMarkdown`、`KuiklyAlbum`、`KuiklyCamera`、`KuiklySQLite`、`KuiklyWebview`、`KuiklyToast`），接着并行访问它们的 GitHub 信息调研用法与平台支持。

有个细节挺有意思：AI 本来在清单里选了`KuiklyMarkdown`来做 Markdown 渲染，但调研后发现`KuiklyChatUI`里的`AiMessageText`已覆盖 AI 消息的 Markdown 渲染场景，于是不再单独引入`KuiklyMarkdown`，最终落地 6 个组件。这就是 Skills 和 Rules 的第一个收益——AI 开始知道什么时候**不该写**。

确定组件清单后，AI 还标出两个需要用 Kuikly 的 Module 机制扩展的能力缺口：

1. 现有`Network`不支持 SSE 长连接，需补一个`SSEModule`
2. 图片发给多模态模型前需统一压缩和 base64 编码，需补一个`ImageModule`

过去用 AI 写代码，最怕它把所有问题都包装成"没问题，我来实现"，然后一路写到编译报错。这次 AI 则表现得像一个资深的 Kuikly 开发者，我知道这是 Kuikly AI 的 Skills 和 Rules 的功劳。它们很好地充当了 AI 的缰绳，哪些用组件、哪些自研、用何种方式组织代码，都被框得很清楚。

方案基本确定

- 聊天主体用 KuiklyChatUI
- Markdown 能力复用 AiMessageText
- 拍照和相册分别用 KuiklyCamera、KuiklyAlbum
- 会话历史用 KuiklySQLite
- 外链打开用 KuiklyWebview
- 轻提示用 KuiklyToast
- SSE 和图片压缩编码用自研 Module 补齐

AI 接着推演了架构、数据结构和交互设计，整体合理，我基本没打断，很快就拿到了第一版开发 plan。

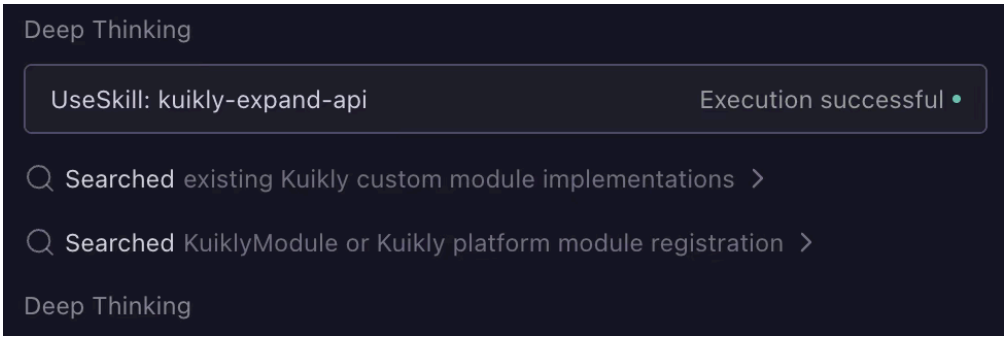


10:20 - 11:10｜编码实现：AI 是怎么把代码写出来的

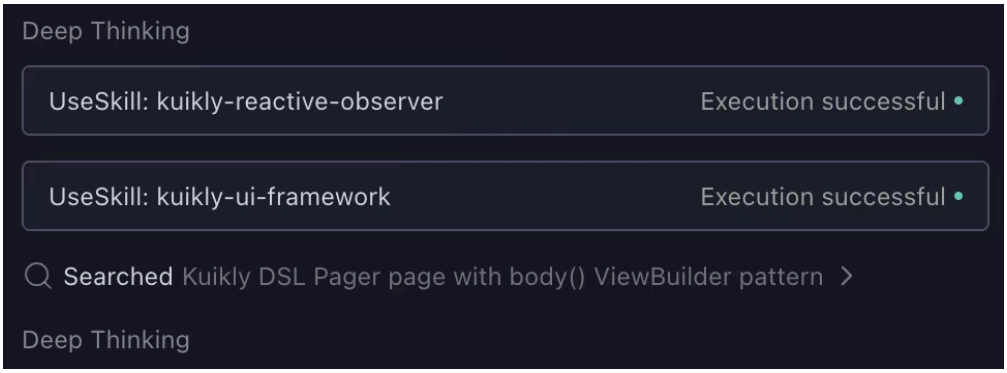
plan 确认后，我基本就让 AI 自己往下跑了，中途没有太多打断。

值得说一下的是它在不同环节调用的技能，这也是这次"AI 真的懂 Kuikly"的关键。

补两个框架没覆盖的能力时，它用的是`[skill:kuikly-expand-api]`——也就是 Kuikly 官方提供的"扩展原生 API / 自定义 Module"技能。`SSEModule`和`ImageModule`这两个跨端 Module 都是靠它生成的：从 commonMain 的接口定义，到三端 native 侧的桥接实现，技能里都有明确的规范可循，AI 不用凭记忆去猜 Kuikly 的 Module 写法。



实现聊天主页面和历史会话列表页面时，它用的是`[skill:kuikly-ui-framework]`和`[skill:kuikly-reactive-observer]`。前者负责 Kuikly DSL 的页面结构和组件用法，后者负责响应式状态——消息列表、流式回复这些需要随状态变化自动重渲染的地方，都是靠`observable`把数据和 UI 绑起来的。



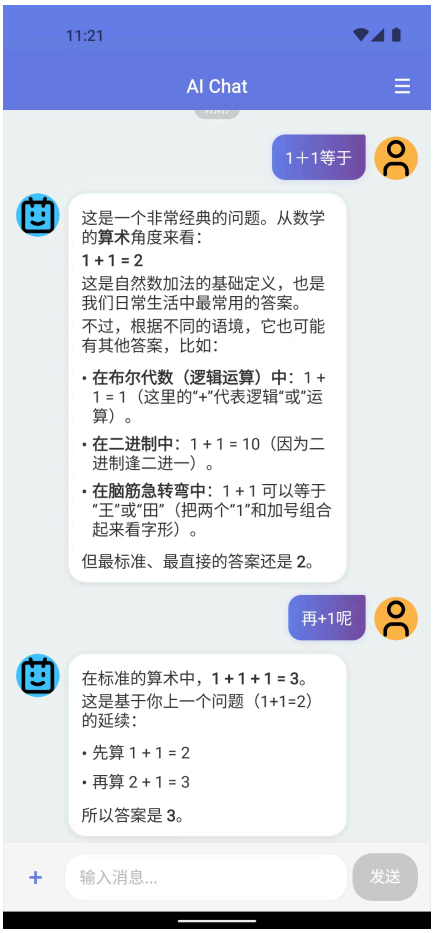
整个过程给我的感觉是：AI 不是在"硬写一种它半懂的 DSL"，而是每到一个环节先加载对应的 Kuikly 技能，再按规范动手。该用哪个组件、该扩展哪个 API、状态怎么绑，基本没走偏。

11:10 - 12:30｜集成自测：上真机测试，发现一个图片加载缺陷

plan 执行完，我先在 Android 真机上跑。

一次编译就成功了。

App 起来，界面干净，输入框、消息列表、发送按钮都在。我发出了第一条文字消息，AI 流式回复正常滚了出来，Markdown 也渲染对了。

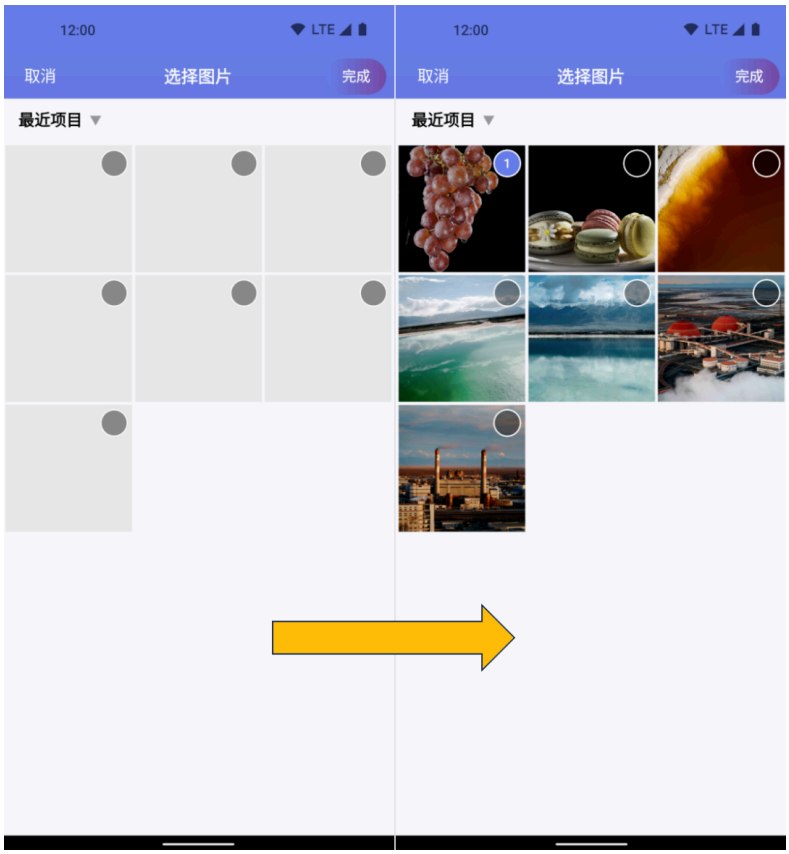


对于一个一码三端、还自带 SSE 长连接和多模态结构的工程来说，第一次真机运行就直接跑通文字链路，这个结果比我预期要好。

文字链路没问题，我接着试图片消息。

图片链路出问题：让 AI 自己定位

点开相册，选图页面正常打开了——缩略图宫格布局也铺出来了，**但每一格的缩略图都加载不出来**，整片宫格只有一个个空白格子，看不到图，自然也没法选。



我没有自己去翻代码，直接把现象和文件甩给 CodeBuddy，让它自己查：

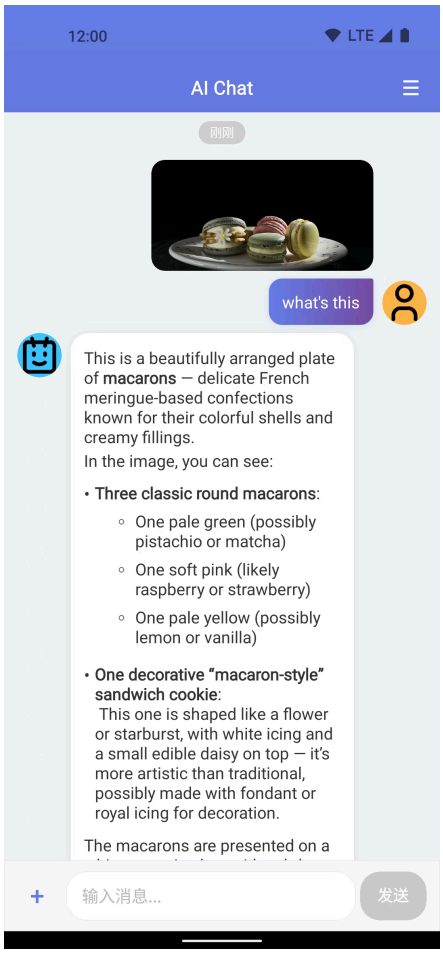
> `@ImagePickerPage.kt` 的图片加载不出来，请添加日志，用adb自行分析原因

AI 按我说的先加日志、再用`logcat`抓日志，用`adb`注入操作复现，顺着日志一步步缩小范围。

很快它就定位到了根因：

> 相册组件给每张缩略图的图片地址是 **content provider 格式**（`content://...`），而模板工程默认实现的 `ImageAdapter` 只处理了 base64、http、assets、file 几种来源，没有认 `content://`这种 URI，所以每张缩略图都解码失败、显示空白。

定位清楚后，它在`ImageAdapter`加上了对 content URI 的识别，重新运行，相册缩略图全部正常显示，选图、发送也都通了。模型能正确识别画面主体并给出对应回答，说明多模态理解链路也跑通了。



这个问题的链路其实很长：从 `KuiklyAlbum` 组件取出缩略图信息，到跨端的 UI 组织，再到原生图片控件上屏，每一步都可能是原因。但因为有 Kuikly AI 的知识库托底，AI 省去了研究框架源码，**我只给出问题现象、相关文件和分析要求，它就能自己加日志、用 logcat + adb 把根因一路查到 `ImageAdapter` 这一层**，就这样轻描淡写地把问题解决了。它再一次扮演了一个资深 Kuikly 开发的角色。

14:00 - 17:30｜迭代优化：把"能用"磨成"好用"

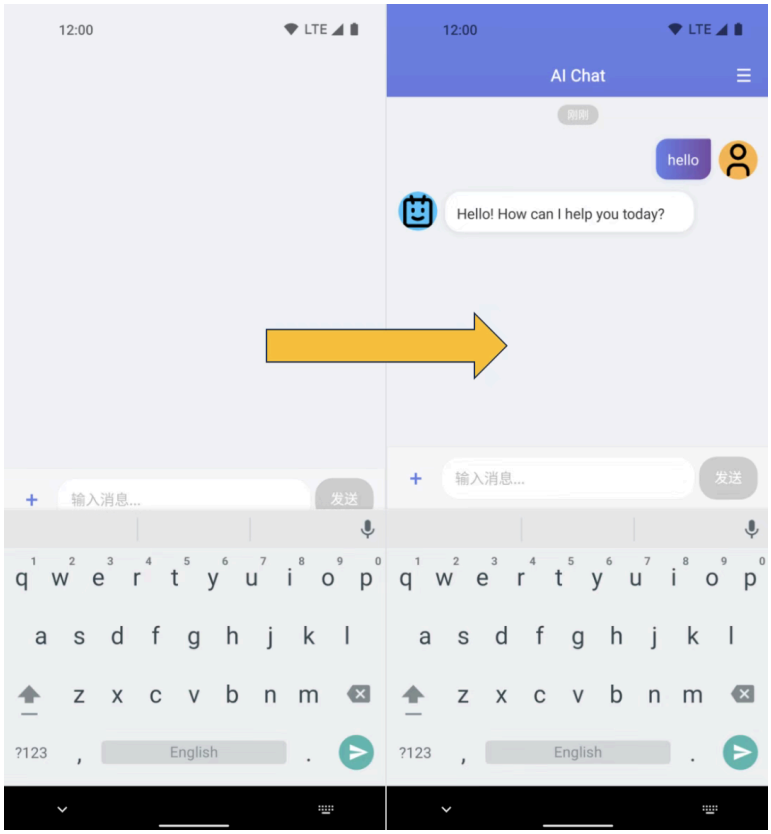
文字和图片链路都通了，剩下的就是细节打磨。产品体验的反复推敲，在原生开发里最磨人，也最容易出现平台不一致。而跨端框架的优势正在于此：一次修改，三端同步改好；配合 Kuikly AI，很多修改进一步简化成了一句 Prompt。

我基本是一条一条把问题丢给 AI，它改完我真机验证，再提下一个。

1）键盘遮挡输入框

最先碰到的是老问题：键盘弹起来，把输入框挡住了。

我让 AI 监听 `keyboardHeight`，用外层容器的 `paddingBottom` 把输入区顶上去。它顺手还处理了一个细节——Kuikly 的键盘事件要挂在 `Input` / `TextArea` 上，而 `ChatSession` 内部的输入框不暴露这个事件，于是它在页面内挂一个代理 Input 承接 Kuikly 的键盘事件回调。这个绕法挺地道，不是我提示的。



2）鸿蒙上新建会话不生效

在鸿蒙端验证时发现一个偏功能性的 bug：新建会话后，历史列表里始终只有那一条旧会话。

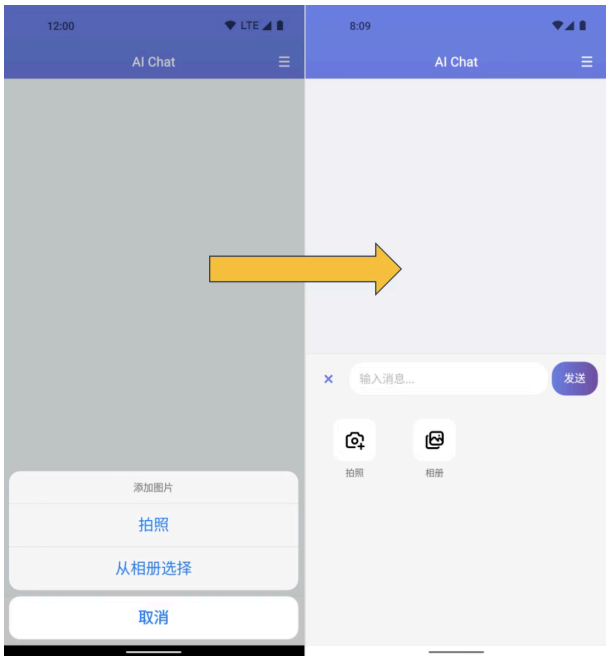
把现象交给 AI，它顺着路由链路定位到根因落在鸿蒙的`RouterAdapter`，这是 Kuikly 处理页面跳转的模块，鸿蒙端默认的`RouterAdapter`没有考虑"先`openPage`再`closePage`"的边界场景。因为有 Kuikly AI 的知识库托底，仅一轮对话就解决了问题。

3）拍照/相册入口：从 ActionSheet 改成宫格按钮

一开始点"+"弹的是系统 ActionSheet。我觉得这不够"聊天 App"，要求改成更常见的、类似微信那种在输入框下方展开的宫格快捷入口。

这一条反而最折腾，一开始 AI 没意识到快捷入口面板和键盘抬升是互斥关系，从面板切到键盘的一瞬间，输入框会抬升"面板 + 键盘"的双重距离，跳一下才落回正常位置。描述澄清了好几轮，AI 才把面板和键盘收敛到同一套底部抬升逻辑里，共享同一个抬升量，保持输入框位置稳定，问题彻底解决。

这也印证了一个共识：AI 从 0 到 1 很快，但从"能用"到"细腻"，还是得靠人一轮轮盯着调。



4）各页面 UI 统一

最后是收口。选图页、网页页这些后加的页面，UI 风格和主聊天页、会话列表页对不齐。我直接让 AI 把`ImagePickerPage`、`WebViewPage`对齐到`ChatPage`、`SessionListPage`的设计规范上。

它先自己读了两个主页面，提炼出一套规范——紫色渐变背景、44dp 透明导航栏、`‹`返回按钮、17sp 白色居中标题、浅色内容区——再把另外两个页面逐项对齐。这种"先归纳现有规范、再套用到新页面"的做法，比我一条条描述样式高效得多。

17:30 - 18:00｜验收复盘：一天，一码三端，一个能用的 AI 聊天 App

到傍晚，这个 App 已经在 Android、iOS、鸿蒙三端真机上都跑通了。

回看一下它最终具备的能力：

- 文本 / 图片消息发送
- 拍照、相册选图
- AI 流式回复（SSE 长连接）
- Markdown 渲染、点链接打开网页
- 多模态图文理解
- 本地会话管理与历史恢复



这不是一个截图演示用的 demo，而是一个交互完整、三端一致、真的能装到手机上聊起来的应用。而这一切，一个人、一天就做完了。

思考总结

Kuikly 解决了什么

如果用传统三端原生开发这样一个 App——iOS、Android、鸿蒙各派一人，设计只做一次，但每块各端相关功能都要写三遍：

模块	工作量
需求 / 架构 / 设计（一次性，不 ×3）	3 人天
聊天 UI（消息列表、气泡、输入框）	2×3 = 6 人天
SSE 流式长连接	1.5×3 ≈ 4.5 人天
相机 / 相册 / 图片处理	2×3 = 6 人天
本地持久化（SQLite / 会话管理）	1×3 = 3 人天
Markdown 渲染 + 网页打开	1.5×3 ≈ 4.5 人天
三端联调 & 打磨	1×3 = 3 人天
合计	**约 30 人天**

换成 Kuikly 一码三端，同样的功能只需一份代码：

阶段	工作量
组件调研 & 接入	2 人天
SSE / 图片 Module 自研（含三端 native 桥接）	2 人天
业务逻辑 & 会话管理	1.5 人天
UI 实现 & 三端验证	2 人天
合计	**约 7.5 人天**

从 30 人天到 7.5 人天，Kuikly 最直接节省的，是这种重复劳动的成本，这个成本可能是开发者的工时，也可能是 AI 的 token 开销。

更深一层节省的，是重复决策。拿"ActionSheet 改成宫格按钮"来说——伴随布局调整，还要连带解决键盘抬升与附件面板的冲突、UI 配色不协调等一连串问题，来回调了好几轮才稳下来。但所有这些轮次，都只在 shared 层发生了一次。如果没有 Kuikly，就要在 Android、iOS、鸿蒙上各来一遍，还要额外确保三端行为对齐。一码多端省掉的，正是这种"同一件事做三遍"的乘数效应。

AI 解决了什么

7.5 人天，是一个 Kuikly 熟手的工作量。而这次接上 AI 后，实际只花了约 7.5 小时：

阶段	实际耗时
环境准备 & 验收复盘	0.5 小时
需求分析 & 方案设计	1 小时
编码实现	1 小时
集成自测 & 调试	1.5 小时
细节打磨	3.5 小时
合计	**约 7.5 小时（1 人天）**

从 7.5 人天压到 1 天，靠的不是 AI 一秒生成几千行，而是 Kuikly AI 让它真的"懂 Kuikly"。这一天最大的感受是：在 Kuikly AI 的 Skills 和 Rules 加持下，AI 不止是一个"会写 Kotlin 的通用模型"，而更像一个熟悉 Kuikly 的资深开发。具体体现在几件事上：

- **自定义 Module 写得很地道。** SSE 和图片压缩这两个框架没覆盖的能力，AI 用`kuikly-expand-api`技能补齐——commonMain 定义接口、三端各实现 native 桥接，结构和官方 Module 如出一辙，不用我去纠正"Kuikly 的 Module 到底该怎么写"。

- **DSL 和响应式理解到位。** 页面结构、组件用法、状态绑定都拿捏得准。消息列表、流式回复这些要随状态自动重渲染的地方，它直接用`observable`把数据和 UI 绑起来，写法很贴合 Kuikly 的范式。

- **自定义组件、扩展原生也不在话下。** 为了监听键盘，它自己加了一个零尺寸的代理`Input`；为了对接相册的`content://`地址，它改对了`ImageAdapter`——这些都是要懂 Kuikly 渲染层和原生接缝才能下手的活。

- **知道什么时候复用、什么时候自己写。** 该用`AiMessageText`就不再造 Markdown 轮子；该扩展能力就老实补 Module；遇到问题（相册缩略图）能自己`logcat + adb`把根因查到`ImageAdapter`，而不是把报错原样丢回来。

这些恰恰是过去 AI 写客户端代码最容易翻车的地方——凭记忆瞎猜私有 API。Kuikly AI 把框架知识喂给模型，相当于补齐了"资深 Kuikly 开发"的上下文，让它在正确的知识边界内工作。

"AI + Kuikly" 是当下客户端开发效率最高的组合之一。

这一天下来，我的体验是，Kuikly 消灭了跨端的重复劳动，AI 消灭了框架层面的样板劳动，剩下留给人的是产品判断、体验打磨、工程把关。说得更具体一点——"AI + Kuikly" 让一个客户端开发，真的有了"一个人顶一个三端小队"的体感。把繁琐交给 Kuikly 和 AI，把判断力和创造力留给自己。

关于 Kuikly

当前Kuikly已经开源，有兴趣和有需要的产品，可以通过以下方式访问 Kuikly 仓库和文档，欢迎Star、Watch与体验：

👉 Github 仓库 | 📚 官方文档

Kuikly框架属于腾讯端服务联盟（tds.qq.com）的重要成员，欢迎关注及了解更多信息：

●腾讯端服务官网：TDS 腾讯端服务

●TDS Framework官网：跨平台框架

「腾讯端服务」服务号	「TDS Framework」公众号	在线咨询
		





阅读 7820

留言

写留言

还没有留言，来写首评